

Relatório Exploratório - Aula 007 (17 - 26)

Slide 17

O slide compara a seleção múltipla tradicional com formas mais modernas de switch e match.

Em C, esquecer o break provoca *fall-through*: a execução continua nos próximos casos:

```
switch (index) {  
    case 1:  
    case 3:  
        odd += 1;  
    case 2:  
    case 4:  
        even += 1;  
    default:  
        printf("Error\n");  
}
```

Para `index = 1`, o programa pode alterar `odd`, alterar `even` e ainda imprimir o erro. O comportamento é válido para a linguagem, mas pode não ser o que o programador pretendia.

Java moderno permite usar switch como expressão, com setas e sem queda implícita:

```
String tipo = switch (dia) {  
    case SAB, DOM -> "fim de semana";  
    case SEG, TER, QUA, QUI, SEX -> "dia útil";  
};
```

Rust usa `match`, que exige casos exaustivos. Se algum valor não for coberto, o compilador exige um caso como `_`. Go encerra cada caso automaticamente, e `fallthrough` precisa ser escrito explicitamente. Python 3.10 introduziu `match` com padrões e guardas.

Slide 18

O `for` de C é apresentado como um laço lógico com inicialização, condição e atualização:

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;  
}
```

Como o corpo e a atualização alteram *i*, a saída é 0 3 6 9. Esse resultado pode surpreender quem olha apenas para a terceira parte do `for`.

As expressões do `for` podem ser omitidas:

```
for (; i <= 100; ) {  
    soma += i++;  
}
```

Em Python, o `for` percorre um iterável e não funciona como o `for` de C:

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2
```

A saída continua sendo 0 1 2 3 4 5 6 7 8 9, porque alterar *i* não altera o iterador produzido por `range`.

As questões de projeto incluem o tipo e o escopo da variável de controle, se ela pode ser alterada no corpo e quando os parâmetros do laço são avaliados.

Slide 19

Este slide mostra laços lógicos, comandos de saída e iteradores. Em Java, um `break` rotulado pode encerrar laços aninhados:

```
externo:  
for (int i = 0; i < matriz.length; i++) {  
    for (int j = 0; j < matriz[i].length; j++) {  
        if (matriz[i][j] < 0) {  
            System.out.println(i + "," + j);  
            break externo;  
        }  
    }  
}
```

O rótulo evita a necessidade de uma variável de controle adicional, mas deve ser usado com moderação para não esconder o fluxo.

Em JavaScript, `for...in` percorre índices ou chaves, enquanto `for...of` percorre valores:

```
const notas = [10, 20, 30];  
let total = 0;  
  
for (const i in notas) total += i;  
console.log(total); // "0012"  
  
total = 0;  
for (const valor of notas) total += valor;  
console.log(total); // 60
```

O primeiro laço também mostra uma conversão implícita: os índices são strings.

O slide retoma o caso dos tocadores Zune, em que um laço sem progresso não atualizava dias quando o valor era exatamente 366 em um ano bissexto. Sem mudança na condição, o laço nunca terminava. O `do-while` também merece atenção porque sempre executa o corpo pelo menos uma vez.

Slide 20

O `goto` é um desvio incondicional. O exemplo baseado no caso "`goto fail`" da Apple mostra como uma linha duplicada pode fazer o fluxo saltar sempre para o rótulo:

```
if ((err = verifica_hash()) != 0)
    goto fail;

goto fail; // executado sempre

if ((err = verifica_assinatura()) != 0)
    goto fail; // nunca alcançado

fail:
return err;
```

O problema não depende apenas do `goto`: a ausência de chaves permite que uma instrução visualmente alinhada pareça pertencer ao `if` quando não pertence.

Dijkstra criticou o uso irrestrito de `goto` por dificultar o raciocínio sobre o fluxo do programa. Algumas linguagens ainda possuem o comando, como C, C++, C# e Go. Outras estruturas, como `break`, `continue` e `return`, são desvios restritos e normalmente deixam a intenção mais clara.

Chaves obrigatórias, como em Go e Rust, e avisos do compilador sobre indentação enganosa ajudam a detectar esse tipo de erro.

Slide 21

Os comandos protegidos de Dijkstra representam escolhas baseadas em guardas:

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Todas as guardas são avaliadas. Se mais de uma for verdadeira, qualquer uma pode ser escolhida de forma não determinística. Se nenhuma for verdadeira, a escolha falha.

Essa ideia está relacionada ao CSP de Hoare e aparece em Go por meio de `select`, usado para aguardar operações de canais:

```
select {
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}
```

Quando os dois canais estão prontos, o Go escolhe um caso de maneira não determinística. Assim, uma execução pode contar 505 ocorrências de a e 495 de b, enquanto outra execução produz valores diferentes.

O ponto exploratório é que não se deve assumir uma ordem fixa quando a própria construção permite escolhas concorrentes.

Slide 22

O primeiro trecho do exercício pede a previsão da saída e a identificação da regra envolvida:

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos) {
    total += p;
}
console.log("Total: " + total);
```

for...in percorre os índices ("0", "1" e "2"), não os preços. Como total começa como número e p é uma string, a soma pode mudar para concatenação. A correção é usar for...of:

```
for (const preco of precos) {
    total += preco;
}
```

O segundo trecho usa Python:

```
def desconto(preco, d=None):
    d = d or 10
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

O valor 0 é falsy, então ele é substituído pelo padrão 10. O resultado é 180, embora a chamada tenha informado desconto zero. Se zero for uma opção válida, a condição deve testar ausência explicitamente:

```
if d is None:
    d = 10
```

Slide 23

O terceiro trecho mostra dois problemas em um switch Java:

```
int dia = 2;
String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

Como não há break, o caso 2 cai nos casos seguintes e o valor final impresso é "inválido". A correção pode usar break em cada caso ou a forma moderna de switch com ->.

O quarto trecho contém um ponto e vírgula após o if:

```
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

O corpo do if é a instrução vazia. Portanto, o printf seguinte é executado sempre, mesmo quando há saldo suficiente. A correção é remover o ponto e vírgula e usar chaves para tornar o corpo explícito.

Slide 24

Os dois últimos exemplos exploram escopo e conversão numérica.

Em Go:

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

O := dentro do bloco declara uma nova variável. Assim, o valor dobrado só existe dentro do if; fora dele, x continua valendo 10. Para alterar a variável externa, seria necessário usar x = x * 2.

Em Java:

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

A saída é 0.0%, porque `acertos / total` é uma divisão inteira e resulta em zero antes da multiplicação e da conversão para `double`. Uma correção é:

```
double taxa = (double) acertos / total * 100;
```

Os exercícios retomam as ideias dos slides anteriores: iteradores, valores falsy, *fall-through*, instruções vazias, escopo e divisão inteira.

Slide 25

O slide de síntese reúne as principais armadilhas da aula:

- **Precedência:** `-2 ** 2` e `1 + 2 + "3"` produzem resultados dependentes da linguagem. Parênteses tornam a intenção explícita.
- **Ordem dos operandos:** expressões com efeitos colaterais podem produzir resultados diferentes em C e Go; Java fixa a ordem da avaliação.
- **Conversões:** C, JavaScript e PHP podem converter silenciosamente. Go e Rust preferem conversões explícitas.
- **Booleanos:** C aceita inteiros como condição, enquanto Java, Go e Rust exigem booleanos.
- **Atribuição:** em C e JavaScript ela pode ser uma expressão; Python, Go e Rust reduzem esse tipo de erro.
- **Seleção:** `switch` sem `break`, `else` pendente e `if (x = y)` são fontes recorrentes de bugs.
- **Laços:** `for...in` pode percorrer chaves, e um laço sem progresso pode nunca terminar.
- **Desvio:** `goto` irrestrito dificulta a leitura; comandos restritos e chaves obrigatórias ajudam.

A conclusão é que conhecer a sintaxe não basta. É necessário entender as regras semânticas da linguagem usada e preferir construções que deixem o fluxo e as conversões evidentes.

Slide 26

As atividades complementares propõem uma autopesquisa sobre:

- expressões em Ruby e Lisp;
- expressões condicionais e erros em expressões;
- implementação de seleção múltipla por tabelas de desvio;
- laços em linguagens funcionais, usando recursão;
- iteradores e blocos de Ruby;
- o texto de Dijkstra, *Go To Statement Considered Harmful*.

Uma entrega sugerida é implementar a mesma decisão em C, Java, Go e Rust, comparando switch, match, queda entre casos, exaustividade e legibilidade.

O próximo assunto anunciado é **subprogramas**, relacionado aos capítulos 9 e 10 de Sebesta. A aula termina reforçando a proposta exploratória: antes de executar, formular uma hipótese sobre a saída e, depois, confrontá-la com o comportamento real da linguagem.